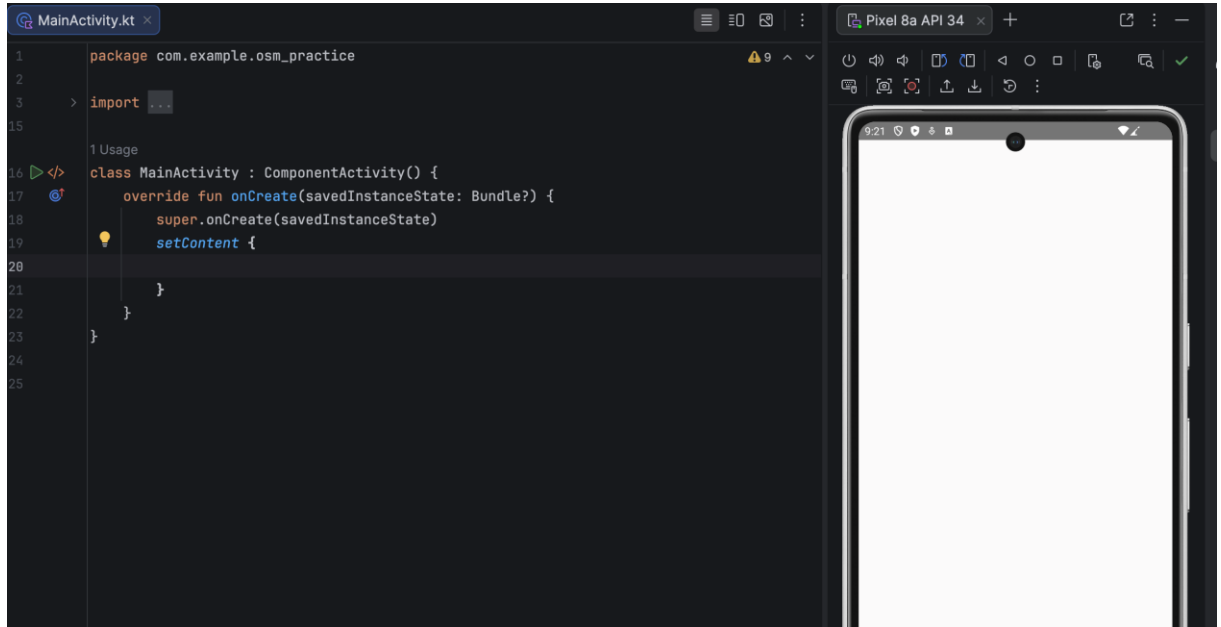


Mobile App Development – working with Maps

- Let's create the empty activity project called OSM-practice with API 34 as the minimum sdk
- Let's clean up the pre-generated Main Activity class



- To work with Open street maps we need to add the following dependency to Gradle configuration:

```
dependencies {  
    implementation("org.osmdroid:osmdroid-android:6.1.20")  
    implementation(platform(dependencyProvider = libs.androidx.compose.bom))  
    implementation(libs.androidx.activity.compose)  
}
```

- And the following permission to the Manifest file:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:tools="http://schemas.android.com/tools">  
    <uses-permission android:name="android.permission.INTERNET"/>  
    <application  
        android:allowBackup="true"  
        android:dataExtractionRules="@xml/data_extraction_rules"  
        android:fullBackupContent="@xml/backup_rules"  
        android:icon="@mipmap/ic_launcher"  
        android:label="@string/app_name"  
        android:roundIcon="@mipmap/ic_launcher_round"  
        android:supportRtl="true"  
        android:theme="@style/Theme.OSM_Practice">
```

- Now let's create our first map so let's add the following function in our MainActivity.kt:

```

@Composable
fun OSMap() {
    val context = LocalContext.current
    AndroidView(
        modifier = Modifier.fillMaxSize(),
        factory = {
            Configuration.getInstance().load(
                ctx = context,
                preferences = context.getSharedPreferences(
                    name = "osmdroid",
                    mode = Context.MODE_PRIVATE
                )
            )
        }
    )
    Configuration.getInstance().userAgentValue = context.packageName
    MapView(context).apply {
        setTileSource(
            TileSourceFactory.MAPNIK
        )
        setMultiTouchControls(true)
        controller.setZoom(15.0)
        controller.setCenter(
            GeoPoint(
                aLatitude = 50.0663,
                aLongitude = 19.9137
            )
        )
    }
}
}
}

```

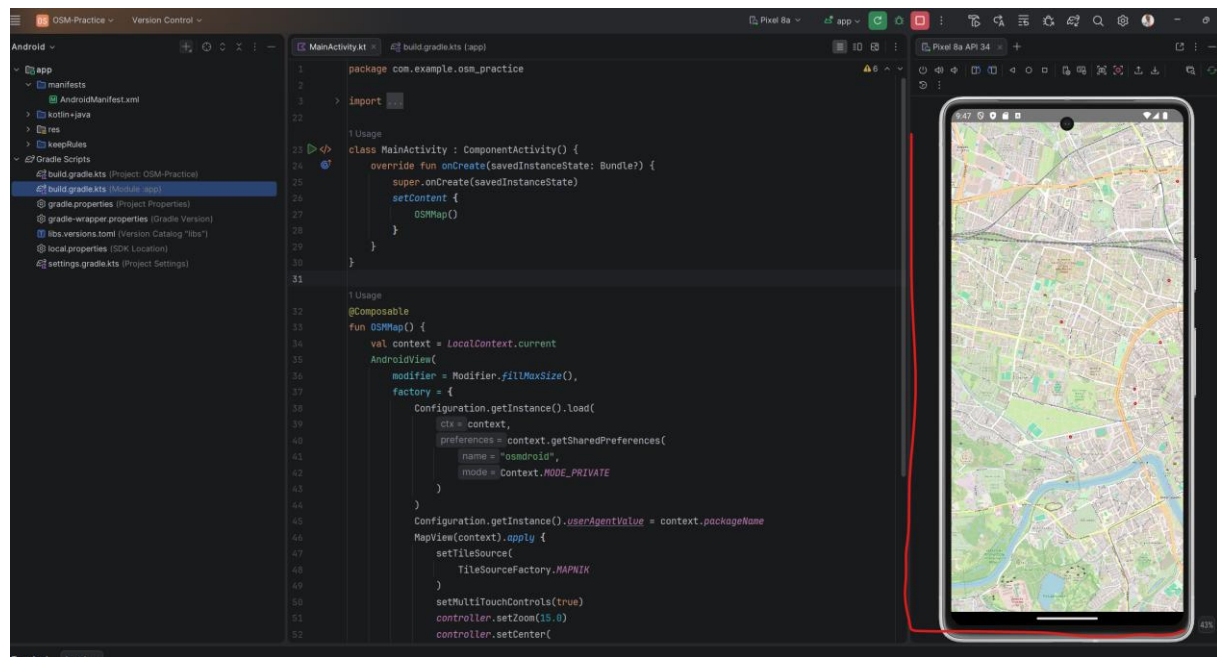
-
- Then, let's call / use this function in our set content:

```

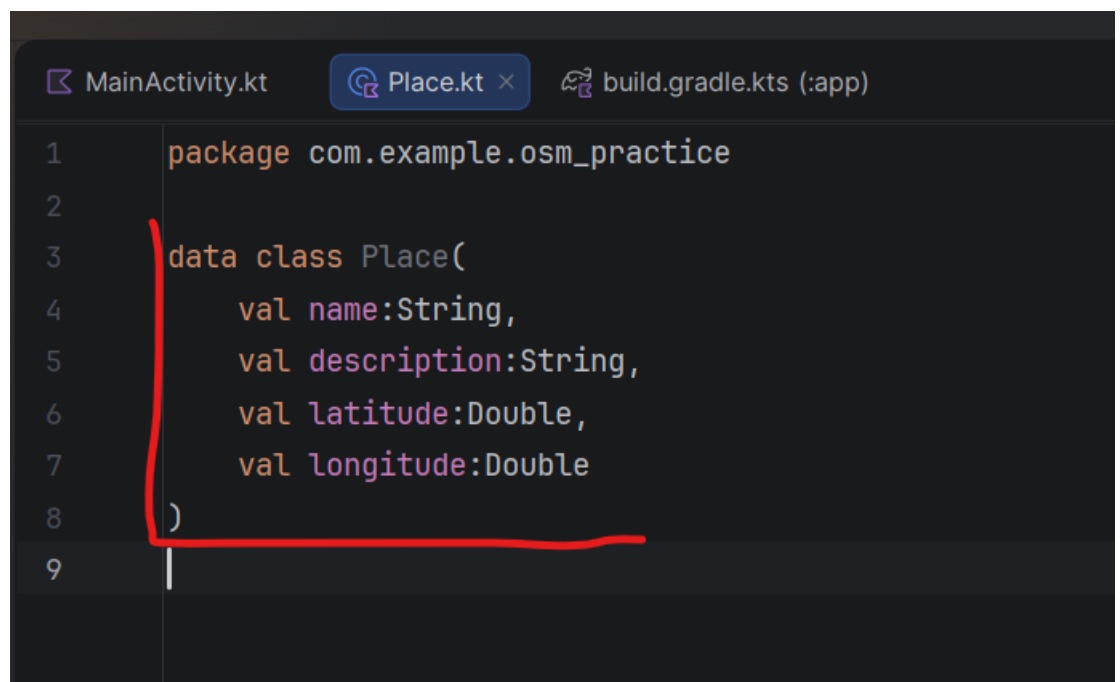
1 Usage
> </>
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            OSMap()
        }
    }
}

```

-
- Let's build our app and we should get:



- Let's now try to work with some points of interests / places, then let create the data class Place:



- Let's add the function for defining the POIs, e.g. like:

```

1 Usage
private fun definePOIPlaces(): List<Place> {
    val places = listOf(
        Place(
            name = "AGH University",
            description = "Faculty of Space Technologies",
            latitude = 50.0663,
            longitude = 19.9137
        ),
    )
    return places
}

```

-
- And let's use this function in our OSMMMap:

```

1 Usage
@Composable
fun OSMMMap() {
    | val places = definePOIPlaces()
    val context = LocalContext.current
    AndroidView(
        modifier = Modifier.fillMaxSize(),
        factory = {
            Configuration.getInstance().load(

```

-
- Let's now add the function for building the marker for the given POI/place, eg. Like this:

```

private fun createPOIMarker(place: Place, mapView: MapView) {
    val marker = Marker(mapView)
    marker.position =
        GeoPoint(
            aLatitude = place.latitude,
            aLongitude = place.longitude
        )
    marker.title = place.name
    | mapView.overlays.add(marker)
}

```

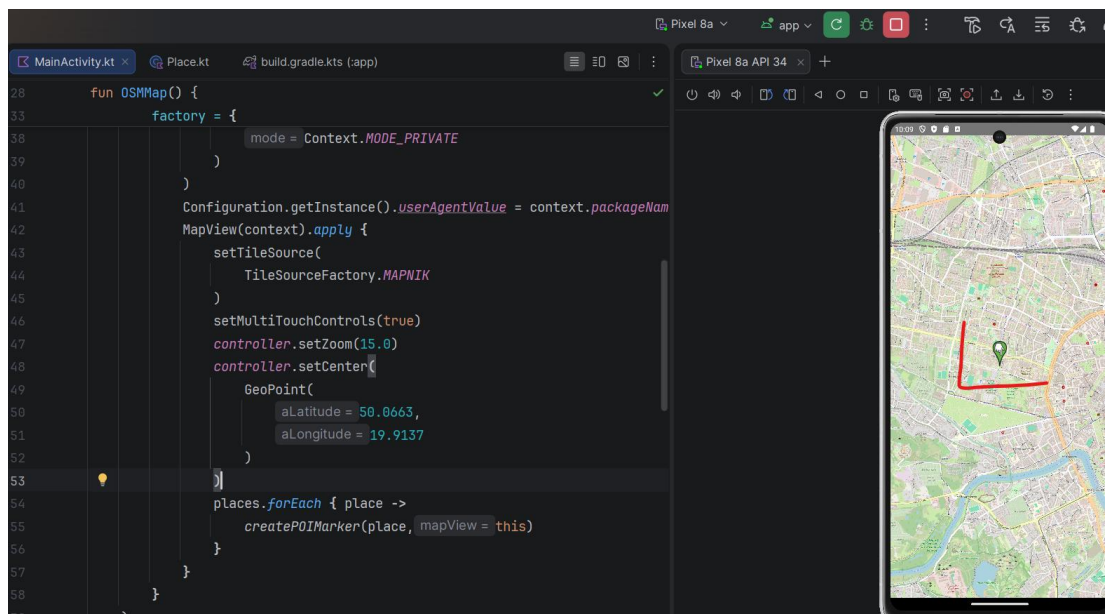
-
- And let's use it in our MapView:

```

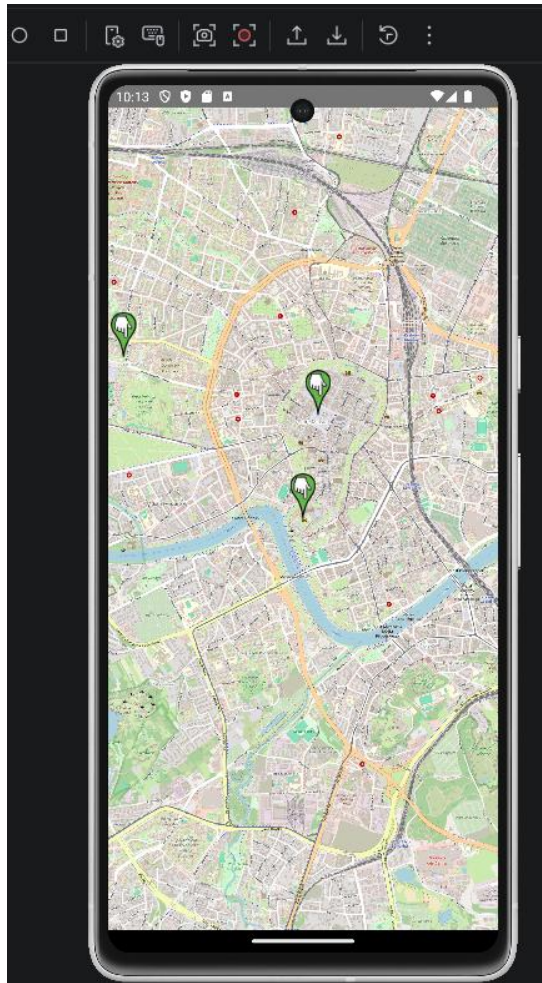
    Configuration.getInstance().userAgentValue = context.packageName
    MapView(context).apply {
        setTileSource(
            TileSourceFactory.MAPNIK
        )
        setMultiTouchControls(true)
        controller.setZoom(15.0)
        controller.setCenter(
            GeoPoint(
                aLatitude = 50.0663,
                aLongitude = 19.9137
            )
        )
        places.forEach { place ->
            createPOIMarker(place, mapView = this)
        }
    }
}

```

- Let's rebuild the app and we should get:



- As for practicing pls add POIs and markers for Krakow Main Square and Wawel Castle, center the map around the Main Square and we should get something like:



○

- As for the next step let's add the descriptions to get something like:



while clicking on the markers

- Before moving on, let's maybe make our code easier to follow by refactoring and getting e.g. something like:

```

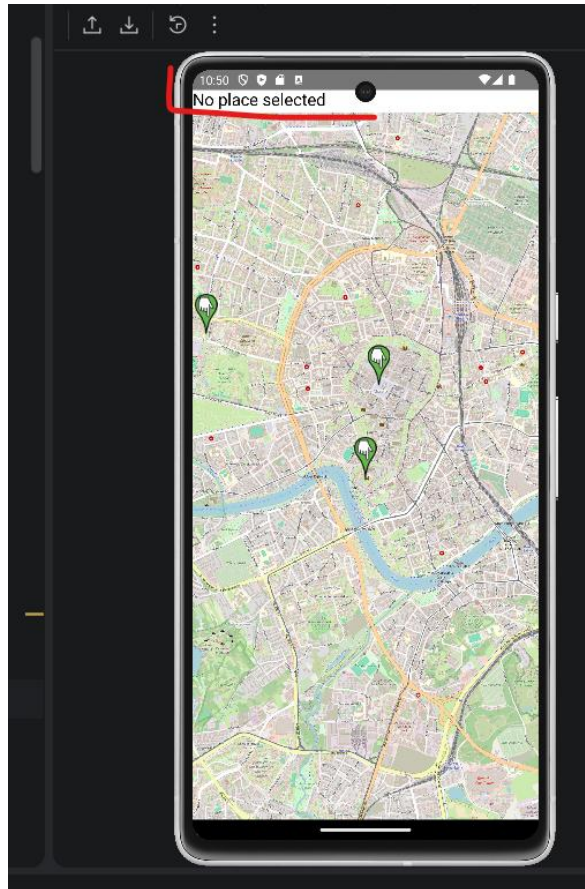
1 Usage
@Composable
fun OSMap() {
    val places = definePOIPlaces()
    val context = LocalContext.current
    AndroidView(
        modifier = Modifier.fillMaxSize(),
        factory = {
            buildMVConfiguration(context)
            createMapView(context, places)
        }
    )
}

```

- Let's now create the alternative createPOIMarker (e.g. called as creatPOIMarkerWithToast) function to get something like:



- Let's now print out information about the POI directly on the APP screen, so let's modify first our UI to get something like:



- Next, let's try to get the following effect while the marker is clicked:



- As the next step let's try to add the custom markers while clicking on the map i.e. something like:



- as the initial state and e.g.



- after clicking on the map (you may start just with adding the markers and then adding the additional information).

○

- When completed pls upload the screenshots / short videocast presenting how your app is implemented and how it works and upload to the upel platform as the solution for the current class and you may switch to working on your final application.